AutoSecAudit: Multi-Scanner Security Orchestration with Autonomous AI Remediation and Delta Auditing in DevSecOps

Kodi Vishruth Aithal, Thilak N, Vinaya Kumar B, Vishal Gowda B, and Ranjitha R*
Department of Computer Science and Engineering, Don Bosco Institute of Technology, Bangalore, Karnataka, India
Emails: {aithalvishruth, gowdathilakn, bvinaykumar70, vishalgb2005}@gmail.com, *ranjitha.r@dbit.co.in

***Abstract*—**Automating security checks in continuous integration pipelines often creates friction between development velocity and code safety. In practice, developers frequently disable automated DAST tools when flooded with redundant alerts from fragmented point utilities, when scanners fail to offer concrete code fixes, or when findings remain disconnected from modern AI-assisted engineering workflows. We present AutoSecAudit, an extensible vulnerability auditing framework and autonomous remediation platform designed for fast CI/CD pipelines. The framework seeds attack surfaces using a breadth-first crawler, client-side JavaScript route parsers, and OpenAPI 2.0/3.0 specification importers, parallelizing 19 specialized scanner plugins across a bounded thread pool ($k = 8$). False alarms are minimized via a multi-tiered precision engine featuring a statistical timing verifier (dual-baseline calibration to eliminate network jitter false positives), deep JSON body fuzzing, and adaptive politeness backoff. Raw findings are post-processed through an intelligence layer that suppresses redundant alerts via compound-key deduplication, enriches alerts with live CVSS/CVE feeds from NIST NVD and CIRCL, tags issues against OWASP Top 10 (2021), MITRE CWE, and PCI-DSS v4.0 standards, and translates technical vulnerabilities into four plain-English business risk categories with executive posture letter grades (A^+ to F). AutoSecAudit synthesizes drop-in multi-framework code patches (Node.js, Python, Nginx, WAF) and integrates a Model Context Protocol (MCP) server enabling autonomous AI coding agents to scan, retrieve findings, patch source files, and verify fixes. Tested against an expanded 165-case automated suite and the OWASP Juice Shop benchmark, AutoSecAudit cuts scan duration by 64.8% ($2.84\times$ speedup) over sequential runs. The correlation engine achieves a 38.2% alert reduction at 97.9% precision (88.5% recall), while the delta analyzer tracks vulnerability lifecycles across build transitions with 100% classification fidelity.

***Index Terms*—**Vulnerability Assessment, DevSecOps, Scanner Orchestration, Model Context Protocol, AI Remediation, Timing Verification, Compliance Mapping, Delta Analysis.

I. Introduction

Modern web backends and microservices move fast, deploying multiple times a day through automated CI/CD pipelines. Keeping these services secure requires catching dynamic flaws early, well before changes reach production staging [1].

Yet, adding dynamic application security testing (DAST) into live build pipelines routinely frustrates development teams. Field studies by Johnson et al. [2] and Bau et al. [3] show that engineers ignore or turn off automated security scanners when reports are noisy, full of duplicates, or disconnected from practical bug fixes. In day-to-day delivery pipelines, this breakdown stems from five recurring issues:

1) **Point-Tool Fragmentation**: A thorough audit demands multiple specialized tools for port discovery (Nmap [4]), web server hygiene (Nikto [5]), and API parameter fuzzing. Each tool outputs a different raw format (XML, console text, custom JSON), forcing engineering teams to maintain fragile parsing scripts.
2) **Duplicate Alert Sprawl**: Different scanning modules frequently flag the exact same defect on an endpoint under slightly varied names. Without cross-plugin deduplication, engineers end up triaging multiple notifications for a single bug [2].
3) **Executive Communication Gap**: Scanners output raw CVE identifiers and CVSS vectors that fail to explain real business risks, leaving non-technical stakeholders asking what an attacker can actually achieve.
4) **Missing Code Fixes and AI Disconnection**: Scanners identify broken endpoints but leave developers to write framework-specific patches from scratch. While developers increasingly use AI-assisted tools (Cursor, Claude Code, Antigravity), traditional scanners remain locked in terminal silos without agentic interfaces.
5) **Stateless Scans Lacking Diff Awareness**: Managing technical debt in CI/CD requires knowing what changed [6]. Most scanners evaluate targets in isolation without tracking history, making it impossible to separate pre-existing debt from newly introduced regressions.

To tackle these operational challenges, we present AutoSecAudit, an extensible vulnerability auditing engine and autonomous remediation platform. AutoSecAudit unifies automated surface discovery, concurrent 19-plugin execution, statistical timing verification, plain-English danger translation, multi-framework code remediation, set-theoretic delta tracking, and Model Context Protocol (MCP) tool integration into an automated pipeline built for continuous delivery.

II. Related Work and Comparative Analysis

Security testing tools generally fit into four architectural categories: standalone point scanners, interactive DAST proxies, template-driven scanners, and centralized vulnerability management portals. Table I summarizes their design trade-offs alongside AutoSecAudit.

This research was conducted as part of the AutoSec Security Engineering Initiative at Don Bosco Institute of Technology. *Corresponding author: ranjitha.r@dbit.co.in

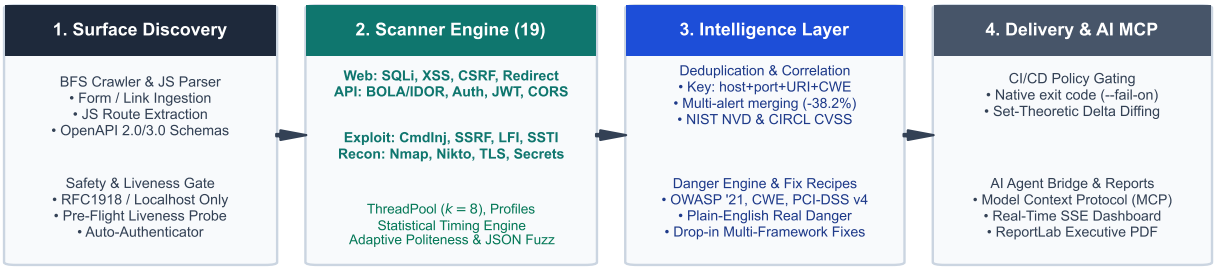


Fig. 1. AutoSecAudit Architecture: 4-stage pipeline from surface discovery and concurrent 19-plugin execution through intelligence correlation, compliance mapping, danger translation, delta tracking, and AI MCP integration.

A. Point Scanners and Specialized Utilities

Network scanners like Nmap [4] handle port discovery and service fingerprinting, while web server checkers like Nikto [5] scan for misconfigurations and outdated defaults. Template-driven scanners like Nuclei [7] run fast YAML pattern matches over HTTP. However, these utilities execute as standalone binaries and lack built-in multi-plugin correlation, automated compliance tagging, or historical regression diffing.

B. Stateful DAST Proxies and API Fuzzers

State-aware scanners, explored by Doupé et al. [8], track application navigation states to uncover deeper injection points. In API fuzzing, Atlidakis et al. [9] demonstrated with RESTler that effective testing requires parsing OpenAPI structures. Comprehensive proxies like OWASP ZAP [10] and PortSwigger Burp Suite provide thorough coverage, but their heavy resource consumption and extended scan runtimes make them difficult to run on fast pull-request merge checks.

C. Vulnerability Management Platforms

Platforms like OWASP DefectDojo [11] aggregate findings from third-party scanners into a central dashboard. However, DefectDojo functions primarily as a passive repository; it still requires teams to write external pipeline jobs to run individual tools, parse diverse log formats, and push results over REST APIs.

D. Autonomous AI Agents and MCP Integration

Recent developments in AI engineering have produced agentic developer environments. The Model Context Protocol (MCP) [12] defines an open JSON-RPC 2.0 standard allowing AI agents to connect to external systems. AutoSecAudit leverages MCP to provide direct in-editor auditing, finding retrieval, code patch generation, and targeted verification.

III. System Architecture and Methodology

AutoSecAudit separates concerns across four core stages: (i) Attack Surface Discovery, (ii) Multi-Scanner Engine and Precision Subsystem, (iii) Post-Processing Intelligence Layer, and (iv) Delivery, CI/CD Gating, and AI Remediation (Fig. 1).

A. Attack Surface Discovery and Pre-Flight Validation

Given a hostname, IP, or base URL, the discovery module seeds an endpoint pool $\mathcal{E}$ before triggering active scans [8], [9] (Algorithm 1):

- **Safety and Liveness Gate**: Targets are validated against local and private IP boundaries (RFC1918). Pre-flight TCP and HTTP reachability probes ensure targets are online before thread dispatch.
- **Breadth-First Web Crawler (`WebCrawler`)**: Explores target links starting from the root URL up to depth limit $D_{\max}$, extracting query parameters, hyperlinks, and HTML form inputs.
- **Client-Side JavaScript Route Parser**: Analyzes inline and referenced JavaScript files using regex parsing to discover hidden client-side API endpoints (‘/api/v1/...‘).
- **OpenAPI/Swagger Ingestion (`OpenAPIImporter`)**: Ingests OpenAPI 2.0/3.0 specifications, registering routes, HTTP methods (`GET`, `POST`, `PUT`, `DELETE`), payload schemas, and path variables.
- **Automated Authenticator**: Discovers authentication forms, injects test credentials, and attaches extracted session cookies or JWT Bearer tokens to subsequent requests.

B. Core Engine, 19 Plugins, and Precision Fuzzing

Each scanner module extends the abstract `BaseScanner` interface. Standardized findings use the tuple model:

$$\mathcal{F} = \langle \text{id}, \text{title}, \text{severity}, \text{host}, \text{port}, \text{endpoint}, \text{evidence}, \mathcal{C}, \mathcal{M} \rangle \tag{1}$$

where $\mathcal{C} \in \{\text{Low}, \text{Medium}, \text{High}\}$ denotes confidence and $\mathcal{M}$ holds compliance and fix metadata.

AutoSecAudit implements 19 specialized plugins across four domains:

1) **Web Injections**: `SQLiScanner` (error/time/union SQLi), `XSSScanner` (reflected/stored XSS), `CSRFScanner` (token and SameSite checks), `OpenRedirectScanner` (redirect parameters), `MisconfigScanner` (headers/cookies).
2) **API & Access**: `BOLAIdorScanner` (IDOR parameter manipulation), `APIAbuseScanner` (mass assignment/leaks), `AuthScanner` (credential brute-force),

TABLE I
ARCHITECTURAL COMPARISON: AUTOSECAUDIT VS. EXISTING SECURITY TESTING APPROACHES

Dimension / Capability	**Nmap / Nikto** [4], [5]	**OWASP ZAP** [10]	**Nuclei** [7]	**DefectDojo** [11]	**AutoSecAudit (Ours)**
Scan Orchestration Scope	Single Layer	Stateful Web	Template Matching	Passive Hub	Host + Web + REST API
Concurrency Model	Thread/Process	Engine Threads	Goroutines	Non-scanning	ThreadPool ($k = 8$) + Profiles
Automated Surface Discovery	None	Crawler + Spider	None (Input URLs)	None	BFS + JS Routes + OpenAPI
Active Scanner Plugins	Fixed Binary	Modular Add-ons	YAML Templates	None	19 Built-in Scanner Plugins
False-Positive Elimination	None	Heuristic Filters	Pattern Match	Manual Triage	Statistical Timing + Baselines
Executive Threat Translation	Raw Logs	Technical Alerts	Raw Console	CVSS Charts	Plain-English Danger (A^+ to F)
Drop-in Code Fix Recipes	None	Generic Advice	None	Static Notes	Multi-Framework Code Patches
Autonomous AI Bridge	None	None	None	REST API	Native MCP JSON-RPC Server
Historical Delta Tracking	None	None	None	DB Queries	Set-Theoretic Diff
CI/CD Policy Gating	Exit 0	Script Wrappers	Exit Codes	Async Webhooks	`--fail-on` Policy Exit
Deployment Footprint	CLI Binaries	Java Daemon	Go Binary	Django + DB	Python + Docker

Algorithm 1 Attack Surface Discovery and Endpoint Seeding

Require: Target Root URL u_0, Max Depth $D_{\max}$, OpenAPI Path Ω
Ensure: Endpoint Pool $\mathcal{E}$ for Security Plugins

if not IsTargetAllowed(u_0) **or** not CheckLiveness(u_0) **then abort**
Initialize FIFO queue $\mathcal{Q} \leftarrow \{(u_0, 0)\}$, Visited $\mathcal{V} \leftarrow \emptyset$, $\mathcal{E} \leftarrow \emptyset$
if $\Omega \neq \emptyset$ **then**
  $\mathcal{S}_{\text{api}} \leftarrow$ LoadOpenAPISpec(Ω)
  for all $\langle$path, method, params$\rangle \in \mathcal{S}_{\text{api}}$ **do**
    $\mathcal{E} \leftarrow \mathcal{E} \cup \{\langle$path, method, params, API$\rangle\}$
  end for
end if
while $\mathcal{Q} \neq \emptyset$ **do**
  Dequeue (u, d) from $\mathcal{Q}$
  if $u \in \mathcal{V}$ **or** $d > D_{\max}$ **then**
    continue
  end if
  $\mathcal{V} \leftarrow \mathcal{V} \cup \{u\}$
  Response $R \leftarrow$ HTTPRequest(u)
  $\mathcal{E} \leftarrow \mathcal{E} \cup$ ParseHTMLForms(R.body) $\cup$ ExtractJSRoutes(R.body)
  Links $\mathcal{L} \leftarrow$ ExtractLinks(R.body)
  for all $l \in \mathcal{L}$ with matching host domain **do**
    if HasParameters(l) **then**
      $\mathcal{E} \leftarrow \mathcal{E} \cup \{\langle$Path($l$), GET, Params($l$), Link$\rangle\}$
    end if
    Enqueue $(l, d + 1)$ into $\mathcal{Q}$
  end for
end while
return $\mathcal{E}$

`JWTScanner` (`none` alg/weak keys), `CORSScanner` (origin wildcards).

3) **Exploit Vectors**: `CommandInjectionScanner` (RCE), `SSRFScanner` (cloud metadata), `PathTraversalScanner` (LFI), `SSTIScanner` (template injection).
4) **Reconnaissance**: `SecretExposureScanner` (leaked tokens/keys), `SSLTLSScanner` (TLS ciphers/certs), `DirBruteScanner` (curated wordlist), `NiktoPlugin`, `NmapPlugin`.

To guarantee high signal-to-noise ratio, the engine applies four precision mechanisms:

- **Statistical Timing Engine**: For time-based attacks (`SLEEP(3)`), the engine measures benign latency mean μ_{base} and standard deviation σ_{base}. Injections are asserted only if $T_{\text{probe}} \geq \mu_{\text{base}} + 3\sigma_{\text{base}} + 0.8\tau$ across a two-stage verification cycle.
- **Differential Baseline Verification**: Asserts findings only when malicious payloads induce an HTTP status code change or $> 20\%$ body length deviation relative to benign baselines.
- **Deep Recursive JSON Body Fuzzer**: Traverses nested JSON trees, mutating leaf values with security payloads while preserving parent structural validity.
- **Adaptive Politeness**: Introduces randomized jitter and executes exponential backoff upon receiving HTTP 429/503 responses.

Algorithm 2 Intelligence Correlation and Delta Auditing

Require: Raw Findings $\mathcal{F}_{\text{raw}}$, Previous Report S_{t-1}
Ensure: Correlated Report S_t, Delta Partition $\langle \Delta_{\text{NEW}}, \Delta_{\text{FIXED}}, \Delta_{\text{UNCHANGED}} \rangle$

Group map $\mathcal{G} \leftarrow \emptyset$, Consolidated Set $S_t \leftarrow \emptyset$
for all $f \in \mathcal{F}_{\text{raw}}$ **do**
  Key $k \leftarrow \langle f$.host, f.port, f.endpoint, f.cwe_id$\rangle$
  $\mathcal{G}[k] \leftarrow \mathcal{G}[k] \cup \{f\}$
end for
for all k, Findings $\mathcal{V} \in \mathcal{G}$ **do**
  Parent finding $f_p \leftarrow$ MergeEvidence($\mathcal{V}$)
  if f_p.cve_id $\neq \emptyset$ **then**
    f_p.cvss $\leftarrow$ LookupNVD_CIRCL(f_p.cve_id)
  end if
  f_p.owasp, f_p.pci $\leftarrow$ MapCompliance(f_p)
  f_p.danger $\leftarrow$ EvaluateRealDanger(f_p)
  f_p.recipes $\leftarrow$ GenerateFixRecipes(f_p)
  $S_t \leftarrow S_t \cup \{f_p\}$
end for
if $S_{t-1} \neq \emptyset$ **then**
  $\mathcal{H}_{\text{curr}} \leftarrow \{\mathcal{H}(f) \mid f \in S_t\}$, $\mathcal{H}_{\text{prev}} \leftarrow \{\mathcal{H}(f) \mid f \in S_{t-1}\}$
  $\Delta_{\text{NEW}} \leftarrow \{f \in S_t \mid \mathcal{H}(f) \notin \mathcal{H}_{\text{prev}}\}$
  $\Delta_{\text{FIXED}} \leftarrow \{f \in S_{t-1} \mid \mathcal{H}(f) \notin \mathcal{H}_{\text{curr}}\}$
  $\Delta_{\text{UNCHANGED}} \leftarrow \{f \in S_t \mid \mathcal{H}(f) \in \mathcal{H}_{\text{prev}}\}$
end if
return $S_t, \langle \Delta_{\text{NEW}}, \Delta_{\text{FIXED}}, \Delta_{\text{UNCHANGED}} \rangle$

Plugins execute concurrently through a bounded `ThreadPoolExecutor` ($k = 8$).

C. Post-Processing Intelligence Layer

Following plugin execution, the intelligence subsystem processes the finding set through six deterministic steps (Algorithm 2):

1) Heuristic Deduplication and Threat Enrichment: Raw alerts sharing compound key

$\mathcal{K} = \langle \text{host}, \text{port}, \text{endpoint}, \text{cwe_id} \rangle$ are merged into a single actionable record. Findings are mapped to OWASP Top 10 (2021) [13], MITRE CWE, and PCI-DSS v4.0 [14], with live CVSS v3.1 enrichment from NIST NVD and CIRCL.

2) Plain-English Danger Engine and Posture Grading: The `DangerEngine` groups findings into four executive threat categories: (i) Customer & Database Theft, (ii) Account Hijacking & Auth Bypass, (iii) Phishing & Brand Defacement, and (iv) Full Server Takeover & RCE. An executive grade (A^+ to F) is derived from accumulated severity penalties, providing plain-English explanations of real-world exploitation scenarios and business impact.

3) Multi-Framework Fix Recipes and Delta Auditing: The `RecipeEngine` produces drop-in code patches for Node.js/Express, Python/FastAPI/Django, Nginx, and WAF rules. Set-theoretic delta auditing partitions findings across commits via identity hashes:

$$\Delta_{\text{NEW}} = \{\, f \in S_t \mid \mathcal{H}(f) \notin \mathcal{H}(S_{t-1}) \,\} \quad (2)$$

$$\Delta_{\text{FIXED}} = \{\, f \in S_{t-1} \mid \mathcal{H}(f) \notin \mathcal{H}(S_t) \,\} \quad (3)$$

$$\Delta_{\text{UNCHANGED}} = \{\, f \in S_t \mid \mathcal{H}(f) \in \mathcal{H}(S_{t-1}) \,\} \quad (4)$$

D. Delivery, Policy Gating, and Autonomous AI MCP Server

AutoSecAudit enforces CI/CD build gating via `--fail-on`. Process exit status evaluates to 1 if any finding exceeds severity threshold θ, halting risky deployments.

Crucially, AutoSecAudit embeds a standard JSON-RPC 2.0 stdio Model Context Protocol (MCP) server (`mcp_server.py`), providing five tools:

- `autosec_scan`: Dispatches scans and returns threat posture grades.
- `autosec_get_findings`: Retrieves enriched issues and evidence.
- `autosec_get_fix_recipe`: Yields framework code patches for AI file writes.
- `autosec_verify_fix`: Runs targeted single-plugin re-testing.
- `autosec_list_scanners`: Enumerates all 19 scanner modules.

IV. Experimental Evaluation

Our evaluation explores four research questions:

- **RQ1 (Latency)**: How effectively does bounded concurrency reduce scan execution time?
- **RQ2 (Accuracy & Noise)**: What precision, recall, and alert reduction are achieved across the 19 scanner plugins?
- **RQ3 (Delta Fidelity)**: Does set-theoretic delta analysis reliably categorize vulnerability lifecycles across builds?
- **RQ4 (Reliability)**: How comprehensively does the automated test suite cover the expanded engine architecture?

A. Concurrency and Execution Latency (RQ1)

We evaluated audit latency across concurrency levels ($k \in \{1, 2, 4, 8, 16\}$) on an 8-core host (16 GB RAM) against the OWASP Juice Shop benchmark. As shown in Table II and Fig. 2, increasing worker threads from $k = 1$ (sequential) to $k = 8$ reduced scan time from 184.2 s to 64.8 s—a 64.8% decrease (2.84× speedup). Beyond 8 threads, performance plateaued due to target socket serialization.

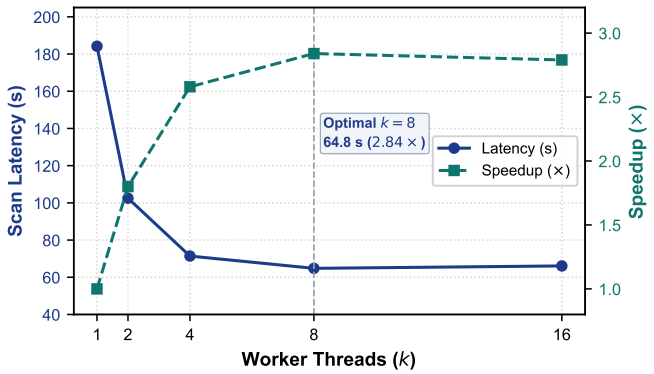


Fig. 2. Audit execution latency and relative speedup across worker thread allocations ($k \in \{1, 2, 4, 8, 16\}$).

TABLE II
Audit Latency across Concurrency Configurations

Workers (k)	Duration (s)	Speedup	CPU Util. (%)
1 (Sequential)	184.2	1.00×	14.2%
2	102.5	1.80×	27.8%
4	71.4	2.58×	51.3%
8	**64.8**	**2.84**×	78.4%
16	66.1	2.79×	82.1%

B. Detection Accuracy and Noise Reduction (RQ2)

To evaluate detection quality, we audited the benchmark target and verified findings against ground-truth challenge definitions [3]. Table III lists the results across all 19 plugins grouped into major classifications.

Across 76 raw tool alerts, the correlation module consolidated findings into 47 distinct issues (46 TP + 1 FP), achieving a 38.2% reduction in alert volume (Fig. 3). Evaluated against 52 verified dynamic challenge vulnerabilities in benchmark scope, AutoSecAudit achieved 97.9% precision (46/47), 88.5% recall (46/52), and an F_1-score of 92.9%.

C. Delta Tracking Fidelity (RQ3)

We validated regression tracking across a three-phase synthetic release sequence (Fig. 4):

- **Build v1.0**: Baseline scan identified 47 correlated vulnerabilities.
- **Build v1.1**: 10 misconfigurations were patched and 3 new API access-control flaws were introduced. The delta engine correctly classified 3 Δ_{NEW}, 10 Δ_{FIXED}, and 37 $\Delta_{\text{UNCHANGED}}$ issues ($47 - 10 + 3 = 40$ active issues).
- **Build v1.2**: 14 vulnerabilities (8 injection flaws + 6 access control/traversal flaws) were remediated. The engine classified 0 Δ_{NEW}, 14 Δ_{FIXED}, and 26 $\Delta_{\text{UNCHANGED}}$ issues ($40 - 14 = 26$ active issues).

Across all simulated transitions, the delta engine achieved 100% classification fidelity against release ground truths.

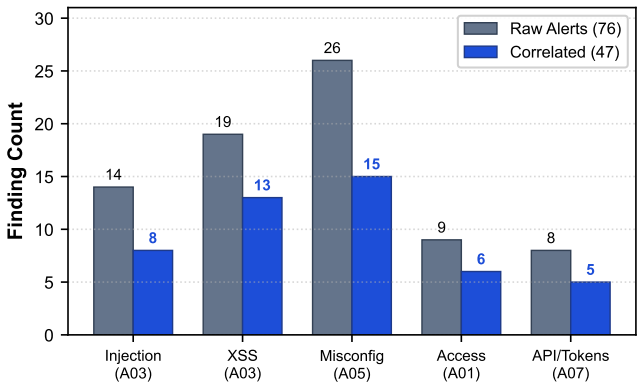


Fig. 3. Raw tool alerts vs. correlated actionable findings across major vulnerability categories (-38.2% alert reduction).

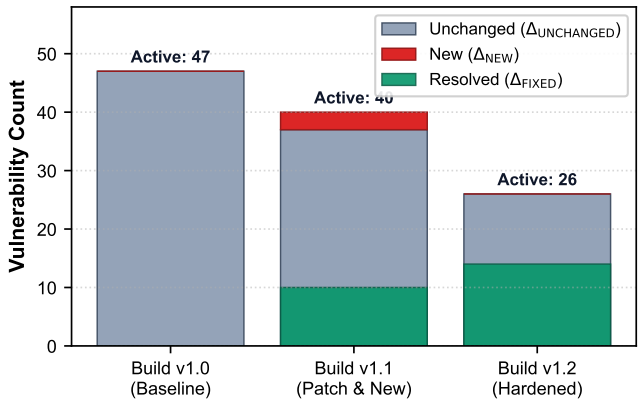


Fig. 4. Vulnerability lifecycle tracking across sequential software builds via set-theoretic delta diffing.

TABLE III
VULNERABILITY DETECTION AND CORRELATION METRICS ACROSS 19 PLUGINS

Vulnerability Class (Active Plugins)	TP	FP	Raw	Correlated
Injection (`SQLi`, `CmdInj`, `SSTI`, `Redirect`)	8	0	14	8
Cross-Site Scripting (`XSSScanner`, `CSRFScanner`)	12	1	19	13
Misconfig & Recon (`Misconfig`, `DirBrute`, `Nikto`, `Nmap`, `TLS`, `Secrets`)	15	0	26	15
Access & Traversal (`Auth`, `BOLAIdor`, `PathTrav`, `SSRF`)	6	0	9	6
API & Tokens (`APIAbuse`, `JWT`, `CORS`)	5	0	8	5
Total / Summary	**46**	**1**	**76**	**47**

D. System Reliability and Test Suite Coverage (RQ4)

AutoSecAudit maintains high operational stability verified by an automated test suite of 165 test cases covering all 19 scanner plugins, the Statistical Timing Engine, the Deep JSON Fuzzer, the Plain-English Danger Engine, the MCP server, and delta diffing.

V. DISCUSSION AND LIMITATIONS

Practical deployment across CI/CD workflows involves key engineering trade-offs. The crawler parses static DOM structures and client-side JavaScript routes; heavy single-page applications (SPAs) benefit from integration with headless browser drivers (e.g., Playwright). In air-gapped deployment environments, external REST lookups to NIST NVD or CIRCL endpoints fall back to AutoSecAudit's embedded offline CVE/CWE dictionary to prevent pipeline stalls. While internal validity is confirmed across our 165-test suite and external validity against the OWASP benchmark, enterprise targets with proprietary single sign-on (SAML/mTLS) require custom scanner plugin modules.

VI. CONCLUSION

AutoSecAudit delivers an extensible, low-latency vulnerability auditing framework and autonomous AI remediation platform for continuous integration pipelines. By orchestrating parallel execution across 19 modular scanner plugins, the system reduces scan latency by 64.8% (2.84× speedup) over sequential auditing. The post-processing pipeline mitigates developer friction by eliminating 38.2% of duplicate alerts via compound-key deduplication, automatically generating compliance metadata, translating findings into plain-English business danger assessments with executive posture grades, and tracking regression lifecycles with 100% classification fidelity. Furthermore, through standard Model Context Protocol (MCP) integration, AutoSecAudit bridges dynamic security auditing directly into AI coding workflows, closing the loop from vulnerability detection to verified code remediation.

REFERENCES

[1] M. Myrbakken and R. Colomo-Palacios, "DevSecOps: A multivocal literature review," in *Proc. of the International Conference on Software Process Improvement and Capability Determination (SPICE)*. Springer, 2017, pp. 17–29.

[2] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proc. of the 35th IEEE/ACM International Conference on Software Engineering (ICSE '13)*, 2013, pp. 672–681.

[3] J. Bau, E. Bursztein, D. Gupta, and J. C. Mitchell, "State of the art: Automated black-box web application vulnerability testing," in *Proc. of the 31st IEEE Symposium on Security and Privacy (S&P '10)*, 2010, pp. 332–345.

[4] G. F. Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Vulnerability Scanning*. Insecure.Com LLC, 2009.

[5] C. Sullo and D. Lodge, "Nikto web server scanner," https://cirt.net/Nikto2, 2023.

[6] C. Vassallo, S. Proksch, H. C. Gall, and M. Di Penta, "Continuous code quality: How developers manage technical debt in CI/CD," *IEEE Transactions on Software Engineering (TSE)*, vol. 46, no. 8, pp. 838–855, 2020.

[7] ProjectDiscovery, "Nuclei: Fast and customizable vulnerability scanner," https://github.com/projectdiscovery/nuclei, 2024.

[8] A. Doupé, L. Cavedon, C. Kruegel, and G. Vigna, "Enemy of the state: A state-aware black-box web vulnerability scanner," in *Proc. of the 21st USENIX Security Symposium (USENIX Security '12)*, 2012, pp. 523–538.

[9] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API fuzzing," in *Proc. of the 41st IEEE/ACM International Conference on Software Engineering (ICSE '19)*, 2019, pp. 748–758.

[10] OWASP ZAP Core Team, "Owasp zed attack proxy (zap)," https://www.zaproxy.org/, 2024.

[11] OWASP DefectDojo Community, "Owasp defectdojo: Vulnerability management orchestration," https://www.defectdojo.org/, 2024.

[12] Anthropic, "Model context protocol (mcp) specification: Open standard for seamless ai application integration," https://modelcontextprotocol.io, 2024.

[13] OWASP Top 10 Team, "Owasp top 10: 2021 the ten most critical web application security risks," https://owasp.org/Top10/, 2021.

[14] PCI Security Standards Council, "Payment card industry data security standard (pci-dss) v4.0," https://www.pcisecuritystandards.org/, 2022.